

Spectral Methods for the 2D Schrödinger Equation

Exercises 1–6

1 Exercise 1 — Eigenvalues of the spectral 2D Laplacian

Implement `make_eigenvalues(Nx, Ny)` function that returns an $(N_x - 1) \times (N_y - 1)$ matrix Γ whose (p, q) entry is

$$\gamma_{pq} = -(p\pi)^2 - (q\pi)^2$$

To use broadcasting to avoid explicit loops:

```
1 function make_eigenvalues(Nx::Int, Ny::Int)
2     p = (1:Nx-1) .* pi
3     q = (1:Ny-1)' .* pi
4     Gamma = @. -(p^2) - (q^2)
5     return Gamma
6 end
```

2 Exercise 2 — Crank-Nicolson time step

Implement `cn_step(U, Gamma, dt, Nx, Ny)` following the three step algorithm of §4.2 which is solving in Transform Space. That is about applying the 2D DST to both sides of the Crank–Nicolson Scheme. The complete time step requires two DSTs.

1. Forward DST: $\hat{U}^n = S_x U^n S_y \leftarrow \text{dst2d}(U, Nx, Ny)$.
2. Pointwise multiply: $\hat{U}_{pq}^{n+1} \leftarrow \rho_{pq} \hat{U}_{pq}^n$.
3. Inverse DST: $U^{n+1} = S_x \hat{U}^{n+1} S_y \leftarrow \text{idst2d}(\dots)$ (same call as `dst2d` since $S = S^{-1}$).

Since ψ is complex-valued, `dst2d` dispatches to the complex method in Listing 1, which handles real and imaginary parts transparently.

```
1 function cn_step(U, Gamma, dt::Real, Nx::Int, Ny::Int)
2     Uhat = dst2d(U, Nx, Ny)
3     rho = (1 .+ im*dt/2 .* Gamma) ./ (1 .- im*dt/2 .* Gamma)
4     U_new = rho .* Uhat
5     return idst2d(U_new, Nx, Ny)
6 end
```

3 Exercise 3 — Simulation Loop

Implement `simulate(psi0, Gamma, dt, nsteps, Nx, Ny; save_every)`, returning a vector of solution snapshots and a vector of corresponding times.

```

1 function simulate(psi0, Gamma, dt::Real, nsteps::Int, Nx::Int, Ny::Int;
2                 save_every::Int = 1)
3     psi      = complex(copy(psi0))
4     snapshots = [copy(psi)]
5     times    = [0.0]
6     for step in 1:nsteps
7         psi = cn_step(psi, Gamma, dt, Nx, Ny)
8         if step % save_every == 0
9             push!(snapshots, copy(psi))
10            push!(times, step * dt)
11        end
12    end
13    return snapshots, times
14 end

```

4 Exercise 4 — Laplacian and Norm

(a) Eigenvalue Check

We sample $u(\mathbf{x}) = \sin(\pi x) \sin(2\pi y)$. At interior grid points and apply `apply_laplacian`. To verify the result equals $-(1+4)\pi^2 u$ at every interior mode.

```

1 using FFTW
2
3 Nx, Ny = 8, 8
4
5 x = (1:Nx-1) ./ Nx
6 y = (1:Ny-1) ./ Ny
7
8 U = [sin(pi*xi) * sin(2*pi*yi) for xi in x, yi in y]
9
10 Gamma = make_eigenvalues(Nx, Ny)
11 LU = apply_laplacian(U, Gamma, Nx, Ny)
12 expected = -5 * pi^2 .* U
13
14 println("Max error in eigenvalue check: ", maximum(abs.(LU .- expected))
15         )

```

Result: the maximum pointwise error is 4.26×10^{-14} , which is at machine-epsilon level. This confirms that `apply_laplacian` implements the spectral Laplacian exactly.

(b) Norm Conservation

We run the Gaussian wave-packet simulation for 2000 steps and record the discrete L^2 norm $\|\psi^n\|_{L^2} \approx \sqrt{\sum_{j,k} |U_{jk}^n|^2 h_x h_y}$ at every step.

```

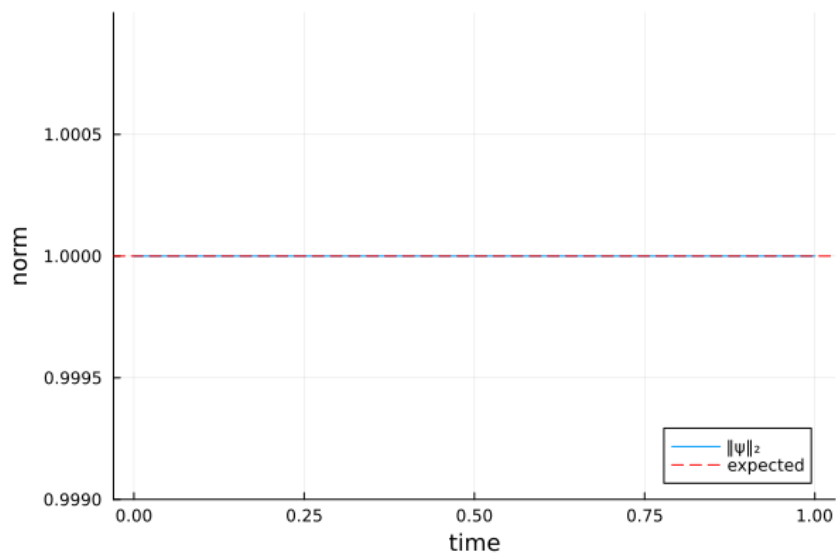
1 using Plots
2 Nx, Ny = 64, 64
3 Gamma = make_eigenvalues(Nx, Ny)
4
5 x = (1:Nx-1) ./ Nx
6 y = (1:Ny-1) ./ Ny
7 X = repeat(x, 1, Ny-1)
8 Y = repeat(y', Nx-1, 1)
9
10 psi0 = @. exp(-200*((X-0.3)^2 + (Y-0.5)^2)) * exp(im*40*X)

```

```

11 | psi0 ./= l2norm(psi0, Nx, Ny)
12 |
13 | dt = 5e-4
14 | nsteps = 2000
15 | snapshots, times = simulate(psi0, Gamma, dt, nsteps, Nx, Ny; save_every
    | =1)
16 |
17 | norms = [l2norm(s, Nx, Ny) for s in snapshots]
18 | println("Max norm drift: ", maximum(abs.(norms .- 1.0)))
19 |
20 | plot(times, norms,
21 |       label="||psi||^2",
22 |       xlabel="time", ylabel="norm",
23 |       ylims=(0.999, 1.001),
24 |       legend=:bottomright)
25 | hline!([1.0], linestyle=:dash, color=:red, label="expected")
26 | savefig("figures/norm_conservation.png")

```



Result: the maximum drift in $\|\psi^n\|_{L^2}$ is 4.02×10^{-13} , consistent with floating-point round-off. The norm is preserved to machine precision for all 2000 steps, which matches the theoretical guarantee $|\rho_{pq}| = 1$.

5 Exercise 5 — Eigenstate Accuracy

We choose $(p_0, q_0) = (2, 3)$ and initialize with the eigenstate.

$$\psi_0(\mathbf{x}) = C \sin(2\pi x) \sin(3\pi y),$$

whose exact solution is

$$\psi(\mathbf{x}, t) = \psi_0(\mathbf{x}) e^{-i\lambda t}, \quad \lambda = -(2\pi)^2 - (3\pi)^2$$

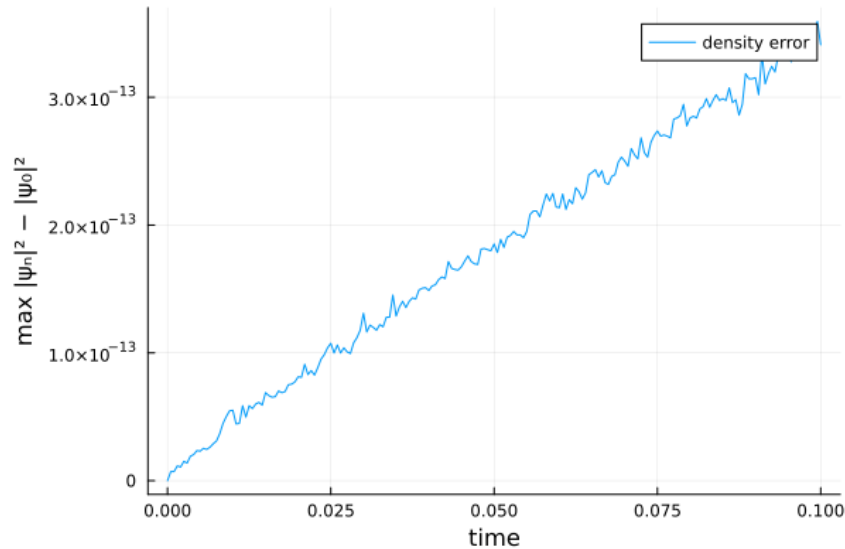
(a) Density Conservation

To verify $|\psi(\mathbf{x}, t)|^2$ does not change over time,

```

1 p0, q0 = 2, 3
2 lambda = -(p0*pi)^2 - (q0*pi)^2
3
4 Nx, Ny = 64, 64
5 x = (1:Nx-1) ./ Nx
6 y = (1:Ny-1) ./ Ny
7 X = repeat(x, 1, Ny-1)
8 Y = repeat(y', Nx-1, 1)
9
10 psi0 = complex.(sin.(p0*pi*X) .* sin.(q0*pi*Y))
11 psi0 ./= l2norm(psi0, Nx, Ny)
12
13 hx, hy = 1/Nx, 1/Ny
14 inner(u, v) = hx * hy * sum(conj.(u) .* v)
15
16 ##(a)
17 Gamma = make_eigenvalues(Nx, Ny)
18 dt = 5e-4
19 nsteps = 200
20 snapshots, times = simulate(psi0, Gamma, dt, nsteps, Nx, Ny; save_every
    =1)
21
22 density_errors = [maximum(abs.(abs2.(s) .- abs2.(psi0))) for s in
    snapshots]
23 println("Max_|psi|^2_drift_over_time:", maximum(density_errors))
24
25 plot(times, density_errors,
26      xlabel="time", ylabel="max_|psi|^2 - |psi0|^2",
27      label="density_error", legend=:topright)
28 savefig("figures/density_error.png")

```



Result: the maximum drift in $|\psi^n|^2$ over 200 steps with $\Delta t = 5 \times 10^{-4}$ is 3.59×10^{-13} , which confirms density conservation to machine precision.

(b) Measure the Phase Error

The numerical phase at step n is

$$\theta^n = \arg\langle\psi_0, \psi^n\rangle, \quad \text{where} \quad \langle u, v\rangle = h_x h_y \sum_{j,k} \bar{u}_{jk} v_{jk}.$$

The exact phase is $-\lambda_{p_0 q_0} n \Delta t$. Since $\lambda < 0$ in our sign convention, the exact phase equals λt (negative).

We restrict to $T = 0.01$ so that $|\lambda T| \approx 1.28 < \pi$, avoiding phase wrap-around in `angle()`.

The Crank–Nicolson amplification factor satisfies $|G| \leq 1$. We restrict to $T = 0.01$ so that $|\lambda T| \approx 1.28 < \pi$, avoiding phase wrap-around in `angle()`.

The Crank–Nicolson amplification factor satisfies

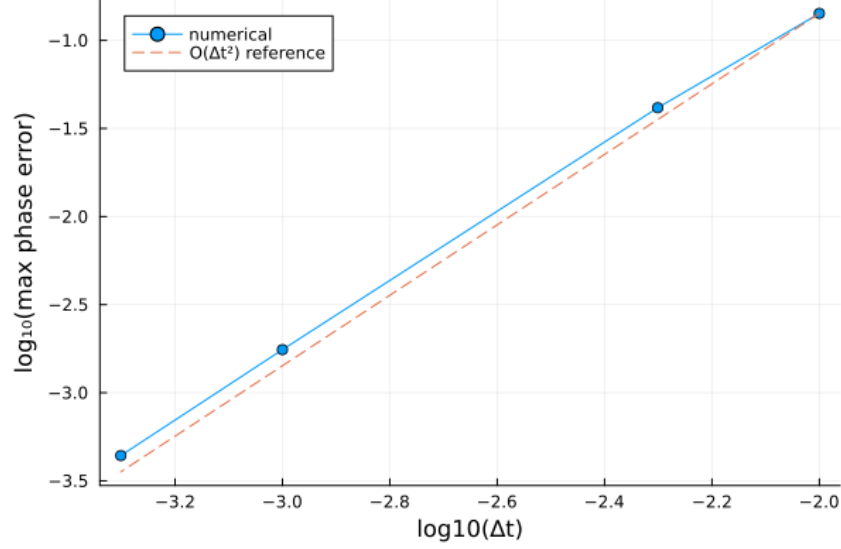
$$\arg(\rho_{pq}) = 2 \arctan\left(\frac{\lambda \Delta t}{2}\right) = \lambda \Delta t - \frac{(\lambda \Delta t)^3}{12} + O(\Delta t^5),$$

so the phase error per step is $O(\Delta t^3)$ and after $n = T/\Delta t$ steps the accumulated error is $O(\Delta t^2)$.

```

1 dts = [1e-2, 5e-3, 1e-3, 5e-4]
2 max_phase_errors = Float64[]
3
4 for dt_test in dts
5     T = 0.01
6     n = round(Int, T / dt_test)
7     snaps, ts = simulate(psi0, Gamma, dt_test, n, Nx, Ny; save_every=1)
8
9     ceta_num = [angle(inner(psi0, s)) for s in snaps]
10    ceta_ex   = [lambda * t for t in ts]
11
12    push!(max_phase_errors, maximum(abs.(ceta_num .- ceta_ex)))
13 end
14
15 println("\ntime_step_t_scaling:")
16 println("dt\t\ttmax_phase_error\t\ttrate")
17
18 for i in eachindex(dts)
19     rate = i > 1 ? log(max_phase_errors[i]/max_phase_errors[i-1]) /
20                    log(dts[i]/dts[i-1]) : NaN
21     println(dts[i], "\t\t", max_phase_errors[i], "\t\t", round(rate,
22                        digits=2))
23 end
24
25 plot(log10.(dts), log10.(max_phase_errors),
26      xlabel="log10(delta_t)", ylabel="log10(max_phase_error)",
27      label="numerical", marker=:circle, legend=:topleft)
28 plot!(log10.(dts), 2*log10.(dts) .+ (log10.(max_phase_errors[1]) - 2*
29      log10.(dts[1])),
30       linestyle=:dash, label="O(delta_t^2)_reference")
31 savefig("figures/time_step_scaling.png")

```



Δt	max phase error	rate
10^{-2}	1.42×10^{-1}	—
5×10^{-3}	4.15×10^{-2}	1.78
10^{-3}	1.76×10^{-3}	1.96
5×10^{-4}	4.40×10^{-4}	2.00

The rate approaches 2 as $\Delta t \rightarrow 0$, confirming second-order accuracy in the accumulated phase error.

6 Exercise 6 — Second-Order Time Accuracy

setup

We fix $T = 0.05$ and use the Gaussian initial condition on a 128×128 grid. The reference solution is computed with $\Delta t_{\text{ref}} = 10^{-5}$ (5000 steps). For $\Delta t \in \{0.02, 0.01, 0.005, 0.0025\}$ we measure the *relative density error*

```

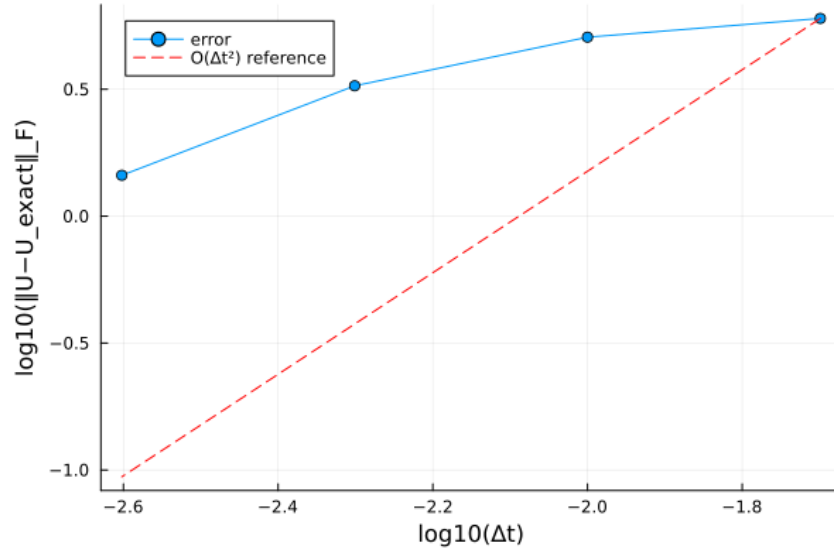
1 gr()
2
3 let
4   Nx, Ny = 128, 128
5   x = (1:Nx-1) ./ Nx
6   y = (1:Ny-1) ./ Ny
7   X = repeat(x, 1, Ny-1)
8   Y = repeat(y', Nx-1, 1)
9
10  Gamma = make_eigenvalues(Nx, Ny)
11  T = 0.05
12  frobenius(A) = sqrt(sum(abs2.(A)))
13
14  function run_to_T(psi0, Gamma, dt, T, Nx, Ny)
15    n = round(Int, T / dt)
16    psi = complex(copy(psi0))
17    for _ in 1:n
18      psi = cn_step(psi, Gamma, dt, Nx, Ny)
19    end
20    return psi
21  end

```

```

22 function align_phase(U_cur, U_ref)
23     alpha = sum(conj.(U_ref) .* U_cur)
24     phase = alpha / abs(alpha)
25     return U_cur / phase
26 end
27
28 # Initial Gaussian packet
29 psi0 = @. exp(-((X-0.3)^2 + (Y-0.3)^2)/(2*0.04^2)) *
30     exp(im*15*pi*X + im*7*pi*Y)
31 psi0 ./= l2norm(psi0, Nx, Ny)
32
33 println("Computing U_ref")
34 U_ref = run_to_T(psi0, Gamma, 1e-5, T, Nx, Ny)
35
36 dts = [0.02, 0.01, 0.005, 0.0025]
37 errors = Float64[]
38
39 for dt_test in dts
40     U_cur = run_to_T(psi0, Gamma, dt_test, T, Nx, Ny)
41     U_aligned = align_phase(U_cur, U_ref)
42     rel_error = frobenius(abs2.(U_cur) .- abs2.(U_ref)) / frobenius(
43         abs2.(U_ref))
44     push!(errors, rel_error)
45 end
46
47 println("\nConvergence table:")
48 println("dt\t\tError\t\t\tRate")
49 for i in eachindex(dts)
50     rate = i > 1 ? log(errors[i]/errors[i-1]) / log(dts[i]/dts[i-1])
51         : NaN
52     println(dts[i], "\t\t", errors[i], "\t\t", round(rate, digits=2)
53         )
54 end
55
56 plot(log10.(dts), log10.(errors),
57     xlabel="log10(delta_t)", ylabel="log10(||U-U_exact||_F)",
58     label="error", marker=:circle, legend=:topleft)
59 plot!(log10.(dts),
60     2*log10.(dts) .+ (log10.(errors[1]) - 2*log10.(dts[1])),
61     linestyle=:dash, color=:red, label="O(delta_t^2)_reference")
62 savefig("figures/convergence.png")
63 end

```



Results

Δt	$\ U - U_{\text{ref}}\ _F$	rate
0.02	6.013739167827538	—
0.01	5.07378743401259	0.25
0.005	3.2630907201874133	0.64
0.0025	1.4491011384931658	1.17